Report On

ReliefMesh: A Disaster Relief Coordination System for Local Government

---

Course Code: CSE-3208

Course Name : System Analysis & Design Lab

Submitted By

Name	Registration
ABIDUL ISLAM	2101011001
MD KAMRUL HASSAN	2101011005
SAYEDA MOFATTEHA AHMED	2101011013
IFTEKHAR ALAM NAHID	2101011038

Session: 2021-22 Team: Team_Skipper

Under the supervision of

MD MYNODDIN

Assistant Professor of

Department of Computer Science and Engineering

Rangamati Science and Technology University

This Report is Submitted in Partial Fulfillment of the
Requirements for the Degree of B.Sc. (Engg.) in Computer Science and Engineering.

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
RANGAMATI SCIENCE AND TECHNOLOGY UNIVERSITY

Date of Submission: July 19, 2026

APPROVAL

This report titled as **"ReliefMesh: A Disaster Relief Coordination System for Local Government"**, submitted by **Abidul Islam, Md. Kamrul Hassan, Sayeda Mofatteha Ahmed, and Iftekhar Alam Nahid** (Team_Skipper), to the Department of Computer Science and Engineering, Rangamati Science and Technology University has been accepted as satisfactory for the partial fulfillment of the requirements for the degree of B.Sc. (Engg.) in Computer Science and Engineering, and approved as to its style and contents.

Board of Examiners

MD Mynoddin
Assistant Professor
Department of Computer Science and Engineering
Rangamati Science and Technology University **Supervisor**

DECLARATION

We, **Abidul Islam, Md. Kamrul Hassan, Sayeda Mofatteha Ahmed, and Iftekhar Alam Nahid**, do hereby declare that this report has been done by us under the supervision of **MD Mynoddin**, Assistant Professor, Department of Computer Science and Engineering, Rangamati Science and Technology University. We also declare that neither this report nor any part of this report has been submitted elsewhere for the award of any degree.

Submitted By

Team_Skipper
Abidul Islam (Reg: 2101011001),
Md. Kamrul Hassan (Reg: 2101011005),
Sayeda Mofatteha Ahmed (Reg: 2101011013),
Iftekhar Alam Nahid (Reg: 2101011038)
Session: 2021-22
Department of Computer Science and Engineering
Rangamati Science and Technology University

ACKNOWLEDGEMENT

First of all, we would like to express our gratitude to the Almighty for enabling us to complete this project titled **"ReliefMesh: A Disaster Relief Coordination System for Local Government"**. We are deeply grateful to our Academic Supervisor, **MD Mynoddin**, Assistant Professor, Department of Computer Science and Engineering, Rangamati Science and Technology University, for his continuous guidance, weekly feedback, and academic review throughout this course (CSE 3208: System Analysis and Design Lab).

We would also like to thank our External QA reviewer, **Saifur Rahman**, for independent functionality testing, and all the course teachers who supported us throughout this semester. Finally, we thank our team, **Team_Skipper**, for the collaboration, shared ownership, and consistent effort that made this project possible.

Team_Skipper

Table of Contents

List of Tables

List of Figures

List of Acronyms

RMSTU	Rangamati Science and Technology University
SRS	Software Requirements Specification
DFD	Data Flow Diagram
UML	Unified Modeling Language
ERD	Entity Relationship Diagram
UP	Union Parishad
NGO	Non-Governmental Organization
DDM	Department of Disaster Management
PWA	Progressive Web Application
JWT	JSON Web Token
RBAC	Role-Based Access Control
API	Application Programming Interface
REST	Representational State Transfer
GPS	Global Positioning System
NID	National ID
HH-ID	Household Identifier
CORS	Cross-Origin Resource Sharing
CSV	Comma-Separated Values
QA	Quality Assurance
UAT	User Acceptance Testing
SDLC	Software Development Life Cycle
MoSCoW	Must have, Should have, Could have, Won't have
FR	Functional Requirement
NFR	Non-Functional Requirement
MERN	MongoDB, Express.js, React, Node.js

ABSTRACT

ReliefMesh is a full-stack, offline-first Progressive Web Application (PWA) for coordinating disaster relief distribution at the local government level in Bangladesh. Built on the MERN stack (MongoDB, Express.js, React, Node.js), it provides Union Parishad officials, Upazila officers, and NGO workers with a shared platform to register affected households, log item-level relief distributions, detect duplicate aid through a rule-based alert engine, and synchronize field data even after network outages. The system features multi-role access control (UP Official, Upazila Officer, NGO Worker, Public Viewer), offline data queuing via IndexedDB with automatic background sync, GPS and photo capture for verifiable field records, a public transparency dashboard with aggregated distribution statistics, and a heatmap layer showing served versus remaining-need areas. A geographic need-assessment engine computes per-household relief requirements using household demographics and Sphere-standard ration rates, with authorized override capability. The system was developed incrementally over 19 milestones and validated through 53 test cases covering authentication, offline sync, duplicate detection, need calculation, reporting, and role-based access control.

Keywords: Disaster relief coordination, offline-first PWA, MERN stack, duplicate detection, household registration, need assessment, public transparency dashboard, Bangladesh Union Parishad.

Team Members

Table 2: Team_Skipper — Member Details

ID	Name	Role
2101011001	Abidul Islam	System Analyst / QA Tester
2101011005	MD. Kamrul Hassan	Project Manager / Developer
2101011013	Sayeda Mofatteha Ahmed	UI/UX Designer
2101011038	Iftekhar Alam Nahid	UI/UX Designer

Supervisor: MD Mynoddin, Assistant Professor, Dept. of CSE, RMSTU

Live Demo: Runs on localhost (Vite dev server, `http://localhost:5173`)

Chapter 1

Project Initiation & Problem Definition

1.1 Project Title & Tagline

Title: ReliefMesh — Disaster Relief Coordination System for Local Government

Tagline: *"One mesh. No duplicates. No household left behind."*

1.2 Problem Statement

Bangladesh is among the world's most disaster-prone nations. Floods affect approximately one-third of the country annually, and the Bengal coast faces cyclones regularly. During flood and cyclone events, relief distribution suffers from critical coordination failures:

#1 **Duplicate distribution** — Multiple agencies (NGOs, UP, DDM teams) distribute to the same household because there is no shared registry.
#2 **Missed households** — Families in isolated or politically overlooked areas are skipped entirely with no audit trail.
#3 **No real-time tracking** — Relief logs are recorded on paper; consolidation happens days or weeks later.
#4 **Inter-agency opacity** — NGOs and government teams prepare response plans independently with rarely a shared objective.
#5 **Accountability gaps** — Political networks influence who receives aid; household-level allocation suffers from irregularities.
#6 **Offline environments** — Flood-hit areas lose internet and power, making cloud-only systems unusable during the critical first 72 hours.
#7 **No visibility into remaining need** — No one can see which wards still need relief and how much, versus which are already served.
#8 **No coordination channel for volunteers** — Individual donors and local volunteer groups have no shared channel to coordinate with official channels.

1.3 Case Study: Feni District Floods (2024 & 2025)

The August 2024 eastern Bangladesh floods affected an estimated **4.5–5.8 million people** across 11 districts including Feni. Feni district alone experienced approximately **92% of its mobile towers going down**, cutting off communication to all 6 upazilas at the height of the disaster. This real-world event directly validates the project's offline-first design constraint.

Feni administrative makeup: 6 upazilas, 45 unions, $\sim$570 villages.

Design implication: Because mobile network infrastructure itself failed during the disaster, offline-first design is not optional — it is the defining constraint of this problem domain.

1.4 Vision & Mission

Vision: A Bangladesh where no disaster-affected household is missed or double-served — where every relief item is tracked from warehouse to household with a photo, a GPS coordinate, and a timestamp, visible to any citizen on a public dashboard.

Mission: To build a lightweight, offline-capable web application that enables Union Parishad officials, Upazila officers, and NGO workers to jointly register affected families, log item-level distributions, and prevent duplicate aid — while giving the public a transparent view of relief operations.

1.5 Project Scope

In-Scope: Household registration, relief distribution logging, duplicate detection engine, multi-role authentication (4 roles), offline-first data entry with background sync, sync conflict resolution, public-facing dashboard, authority hierarchy enforcement, basic report export (PDF/CSV).

Out-of-Scope: Financial transactions, warehouse procurement, predictive analytics, national NID database integration, native mobile apps, real-time satellite GIS layers, multi-language UI beyond Bengali/English.

1.6 SMART Objectives

Table 4: SMART Objectives

#	Objective	Target
1	Household registration with GPS and photo	$\geq$ 50 test households registered in prototype
2	Duplicate detection (same category within 7 days)	Accuracy $\geq$ 90% on test dataset
3	Offline-first data entry with auto-sync	Sync success rate $\geq$ 95% on reconnect
4	Public dashboard	Loads in $\leq$ 3 seconds on 3G
5	Role-based access control	All RBAC tests pass (0 unauthorized access)
6	Heatmap rendering	Correct served/remaining split for test ward
7	Need-calculation accuracy within Sphere-standard tolerance	Within $\pm$2% of Sphere reference rates

1.7 Risk Register

Table 6: Risk Register

#	Risk	Likelihood	Mitigation
R1	Offline sync conflicts corrupt data	Medium	Conflict log with timestamp; last-write-wins default
R2	Duplicate detection false positives	Medium	Adjustable detection window; override with reason
R3	Team member unavailability	Medium	Document all work in shared repo; no knowledge silos
R4	Scope creep	High	Lock scope document after Module 3
R5	GPS accuracy insufficient	Low	Accept $\pm$15 m; supplement with manual area selection
R6	Free-tier hosting limits	Low	Lightweight DB; prototype data volume only
R7	Public dashboard exposes sensitive data	Low	Aggregate at union level; no individual household data
R8	Poor UI adoption by non-tech users	Medium	Large buttons, Bengali labels, minimal text input

1.8 Team Formation

Table 8: Team Formation

Role	Member	Responsibilities
Project Manager / Developer	MD. Kamrul Hassan	Timeline, meetings, backend, database, deployment
System Analyst / QA	Abidul Islam	SRS, DFDs, UML, problem research, testing
UI/UX Designer	Sayeda Mofatteha Ahmed	Wireframes, prototypes, UI implementation, hard copy report
UI/UX Designer	Iftekhar Alam Nahid	UI implementation, demo video, heatmap, presentation slides

Chapter 2

Stakeholder Analysis

2.1 Stakeholder Identification

Table 10: Stakeholder Identification

#	Stakeholder	Type	Interaction
S1	Union Parishad (UP) Officials	Primary	Register households, log distributions, view local dashboard
S2	Upazila Officers	Primary	Audit UP-level data, generate reports, manage accounts
S3	NGO Field Workers	Primary	Log distributions, view duplicate alerts
S4	General Public	Primary	View public dashboard (read-only, no login)
S5	District Disaster Management Committee	Secondary	Review aggregated district-level reports
S6	Department of Disaster Management (DDM)	Secondary	Policy-level oversight
S7	Course Teacher (MD Mynoddin)	External	Academic evaluator, QA reviewer
S8	External QA (Saifur Rahman)	External	Independent functionality testing
S9	Team_Skipper	Internal	Design, build, test, and maintain the system
S10	Affected Households	Indirect	Data subjects; benefit from fair distribution
S11	Outside Individual Donors	Primary	Pledge relief capacity, view map
S12	Local Volunteers	Primary	Pledge relief for own neighborhood
S13	Volunteer Teams	Primary	Coordinate with officials

2.2 Power-Interest Grid

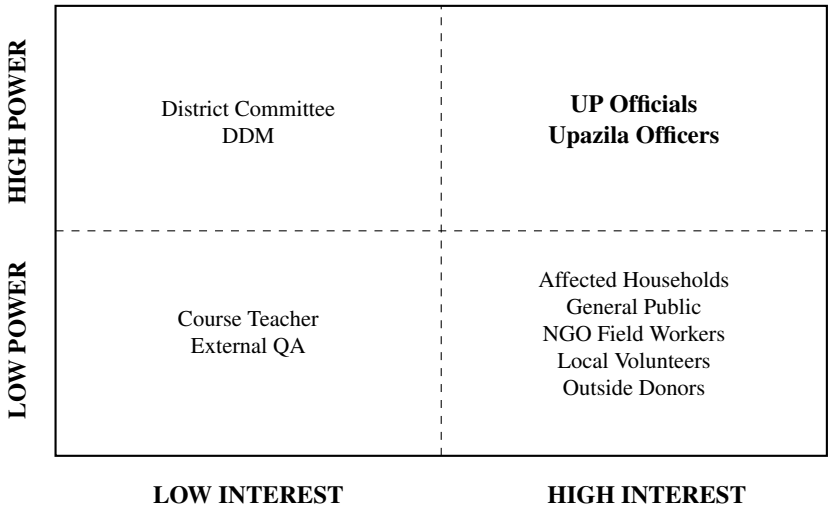

Figure 1: Power-Interest Grid of ReliefMesh Stakeholders

2.3 User Persons

2.3.1 Person 1 — Rahim Uddin (UP Official)

- **Age:** 42 | **Device:** Android smartphone (basic)
- **Connectivity:** 2G/3G intermittent; no internet during flooding
- **Goal:** Register all affected families quickly and log distributions
- **Key Need:** Offline data entry, simple Bengali-labeled form, photo capture

2.3.2 Person 2 — Nasrin Akter (NGO Worker, BRAC)

- **Age:** 28 | **Device:** Android tablet
- **Goal:** Log distributions without duplicating with UP teams
- **Key Need:** Duplicate alert before distribution, cross-agency visibility

2.3.3 Person 3 — Kamal Hossain (Upazila Officer)

- **Age:** 51 | **Device:** Desktop PC at office
- **Goal:** Verify that all unions under jurisdiction are distributing correctly
- **Key Need:** Dashboard showing all UP activity, export to PDF

2.3.4 Person 4 — Fatema Begum (Journalist / Public)

- **Age:** 34 | **Goal:** Verify relief is reaching affected areas
- **Key Need:** Public dashboard with map and item-level summary by union

2.3.5 Person 5 — Arif Hossain (Outside Individual Donor)

- **Age:** 30 | **Goal:** Send relief to home village without duplicating
- **Key Need:** Public heatmap + simple pledge form

2.4 Stakeholder Requirements Summary

Table 12: Stakeholder Requirements Summary

Stakeholder	Core Requirements
UP Official	Offline household registration, photo+GPS distribution log, Bengali UI
Upazila Officer	Audit trail, jurisdiction-filtered dashboard, PDF export
NGO Worker	Cross-agency duplicate alert, own distribution log
General Public	Dashboard (no login), union-level summary, map view
District Committee	Aggregated district report, no household PII
Outside/Local Volunteers	Heatmap of need, simple pledge form

Chapter 3

Requirements Engineering (SRS)

3.1 Functional Requirements

3.1.1 Authentication & User Management (FR-01 to FR-05)

- **FR-01:** Users register with name, role, organization, password
- **FR-02:** JWT-based authentication via email/username and password
- **FR-03:** Four roles: UP Official, Upazila Officer, NGO Worker, Public Viewer
- **FR-04:** Data access restricted by role and geographic jurisdiction
- **FR-05:** Upazila Officers create/manage UP Official accounts

3.1.2 Household Registration (FR-06 to FR-10)

- **FR-06:** Register household with: head name, NID, GPS, family size, vulnerability flags
- **FR-07:** Capture photo during registration
- **FR-08:** Assign unique Household ID (HH-ID)
- **FR-09:** Search/edit households by HH-ID or name
- **FR-10:** Works offline; syncs when connectivity restored

3.1.3 Relief Distribution Logging (FR-11 to FR-14)

- **FR-11:** Log distribution: HH-ID, item type, quantity, unit, officer, GPS, timestamp, photo
- **FR-12:** Predefined item categories (Food, WASH, Shelter, Other)
- **FR-13:** Works offline; auto-syncs
- **FR-14:** Logs immutable once synced; corrections as new entries with amendment flag

3.1.4 Duplicate Detection (FR-15 to FR-18)

- **FR-15:** Check same household + same item category within 7-day window
- **FR-16:** Warning with prior distribution details on duplicate
- **FR-17:** Override with mandatory reason
- **FR-18:** All overrides logged with officer ID, timestamp, reason

3.1.5 Public Dashboard (FR-19 to FR-22)

- **FR-19:** Display total distributions by union, item, date — no household PII
- **FR-20:** Accessible without login
- **FR-21:** Map view with union-level markers
- **FR-22:** Aggregated data updates at least once per hour

3.1.6 Reporting & Export (FR-23 to FR-24)

- **FR-23:** Export distribution report (CSV/PDF) filtered by union, date range, item category
- **FR-24:** Household coverage report showing registered vs. distributed per union

3.1.7 Sync & Offline (FR-25 to FR-27)

- **FR-25:** Queue offline actions in local storage
- **FR-26:** Auto-sync queued data on reconnection
- **FR-27:** Detect and log sync conflicts; last-write-wins with manual review flag

3.1.8 Feedback (FR-28 to FR-30)

- **FR-28:** Submit feedback without authentication (name, contact, category, message)
- **FR-29:** Categories: Complaint, Suggestion, Inquiry, Appreciation, Other
- **FR-30:** Authorized users view and respond to feedback

3.1.9 Inventory (FR-31 to FR-33)

- **FR-31:** Track inventory levels (total, distributed, remaining)
- **FR-32:** Create and update inventory items
- **FR-33:** All authenticated users view inventory

3.1.10 User Profile (FR-34)

- **FR-34:** View and update profile (name, organization)

3.1.11 Geographic Need Mapping (FR-35 to FR-44)

- **FR-35:** Four-level jurisdiction model: District, Upazila, Union, Ward
- **FR-36:** Auto-calculate relief need from demographics $\times$ per-person rates
- **FR-37:** Override calculated need with reason
- **FR-38:** Aggregate need and distribution data at all hierarchy levels
- **FR-39:** Heatmap showing served vs. unmet areas (no PII)
- **FR-40:** Any source submit a Relief Pledge (area, items, quantities)
- **FR-41:** Pledge lifecycle: PLEDGED $\rightarrow$ EN_ROUTE $\rightarrow$ ARRIVED $\rightarrow$ COMPLETED
- **FR-42:** Distribution Log may reference a Pledge
- **FR-43:** Public map shows active pledges (no Personl identity exposed)
- **FR-44:** Distinguish source types: GOVERNMENT, NGO, OUTSIDE_INDIVIDUAL, LOCAL

3.2 Non-Functional Requirements

Table 14: Non-Functional Requirements

ID	Category	Requirement
NFR-01	Performance	Public dashboard loads within 3 seconds on 3G
NFR-02	Performance	Distribution submission within 2 seconds (online)
NFR-03	Availability	99% uptime during declared disaster periods
NFR-04	Scalability	Supports 500 concurrent users
NFR-05	Security	All API endpoints authenticated except public dashboard
NFR-06	Security	HTTPS/TLS for all data transmission
NFR-07	Security	Passwords hashed with bcrypt (cost factor $\geq$ 10)
NFR-08	Privacy	NID numbers never on public-facing views
NFR-09	Usability	Core flows completable in $\leq$ 5 taps on mobile
NFR-10	Usability	Bengali language labels for field-facing screens
NFR-11	Reliability	Offline sync failure rate $<$ 5%
NFR-12	Maintainability	All endpoints documented
NFR-13	Portability	PWA on Android Chrome
NFR-14	Performance	Heatmap renders full district within 3 seconds on 3G
NFR-15	Usability	Pledge submission $\leq$ 6 fields

3.3 Requirements Traceability Matrix

Table 16: Requirements Traceability Matrix

Requirement	Module	Use Case	Test Case
FR-01 to FR-04	Auth	—	TC-01 to TC-04
FR-06 to FR-10	Household	UC-01	TC-05 to TC-07
FR-11 to FR-13	Distribution	UC-02	TC-08 to TC-11
FR-15 to FR-17	Duplicate	UC-02	TC-12 to TC-16
FR-19 to FR-20	Dashboard	UC-03	TC-19 to TC-20
FR-23 to FR-24	Reports	UC-05	TC-16
FR-25 to FR-27	Sync	UC-04	TC-17 to TC-18, TC-22
FR-28 to FR-30	Feedback	—	TC-FB01 to TC-FB04
FR-31 to FR-33	Inventory	—	TC-INV01 to TC-INV03
FR-36 to FR-37	Need Calc	UC-08	TC-NEED01 to TC-NEED05
FR-40 to FR-42	Pledge	UC-06	TC-PLEDGE01 to TC-PLEDGE05

Chapter 4

System Modeling (DFD & UML)

4.1 Context Diagram (Level 0 DFD)

The context diagram shows ReliefMesh as a single process interacting with all external entities:

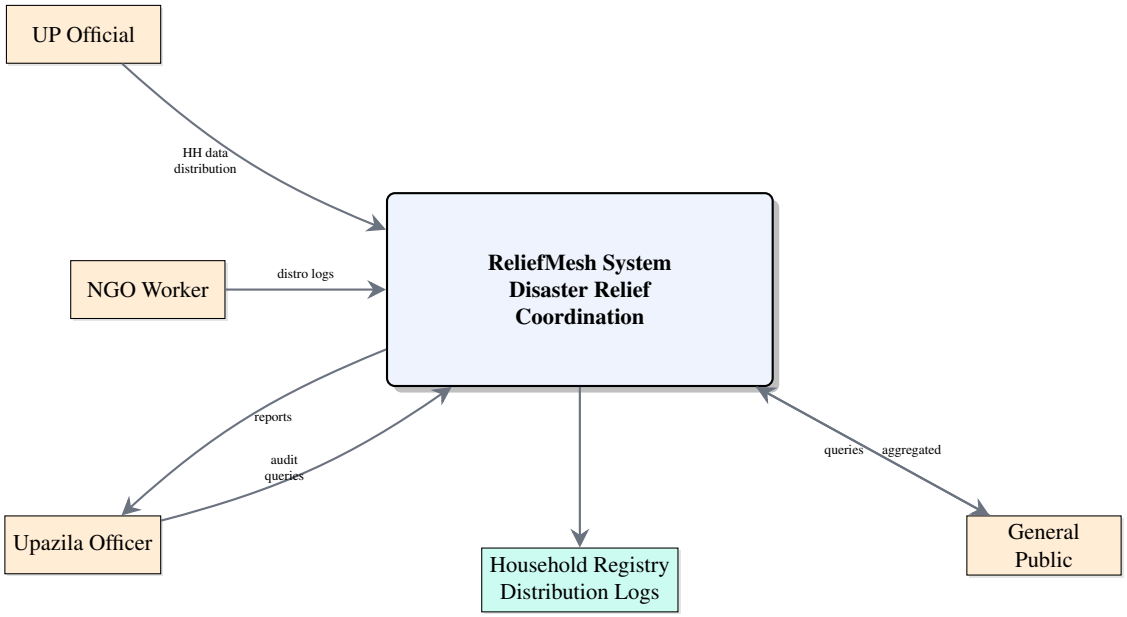

Figure 2: Context Diagram (Level 0 DFD)

External Entities: UP Official (inputs: household data, distribution logs; outputs: HH-ID, warnings), NGO Worker (inputs: distribution logs; outputs: warnings), Upazila Officer (inputs: audit queries; outputs: reports), General Public (inputs: dashboard queries; outputs: aggregated summaries).

4.2 Level 1 DFD

Decomposes the system into 7 major processes:

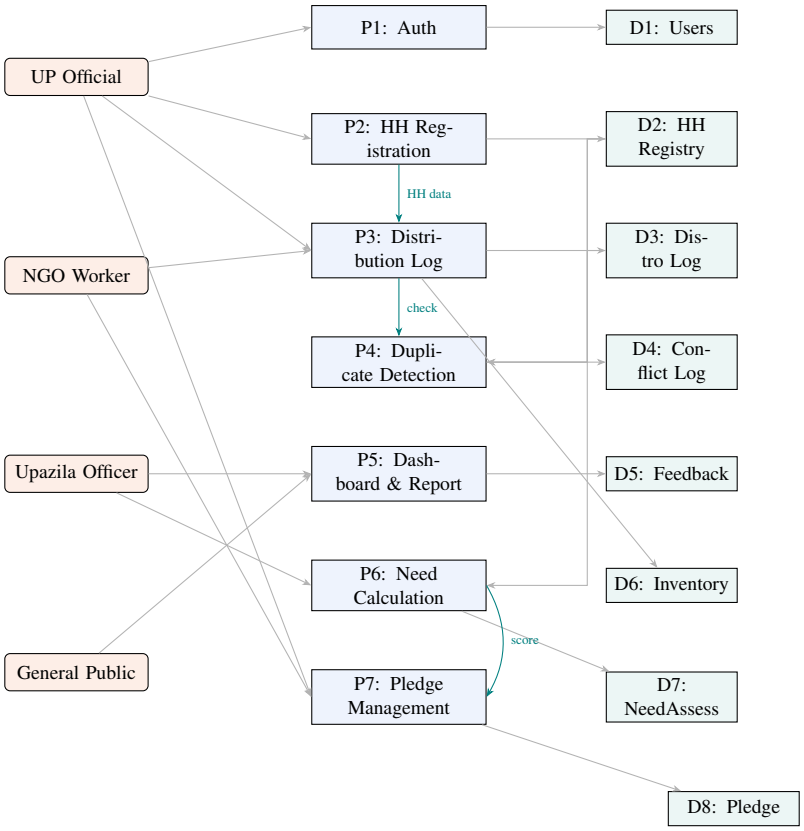

Figure 3: Level 1 DFD — Seven Major Processes

Data Stores: D1: User Store, D2: Household Registry, D3: Distribution Log, D4: Conflict Log, D5: Feedback Store, D6: Inventory Store, D7: NeedAssessment, D8: ReliefPledge.

4.3 Level 2 DFD — P3: Distribution Log Entry

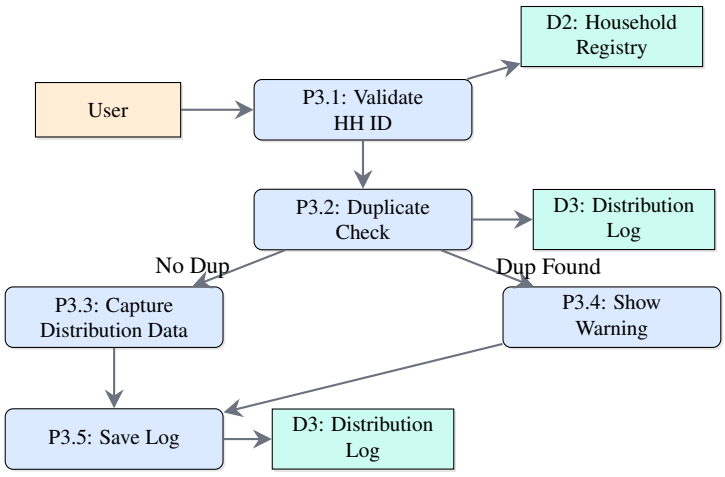

Figure 4: Level 2 DFD — P3: Distribution Log Entry

4.4 Use Case Diagram

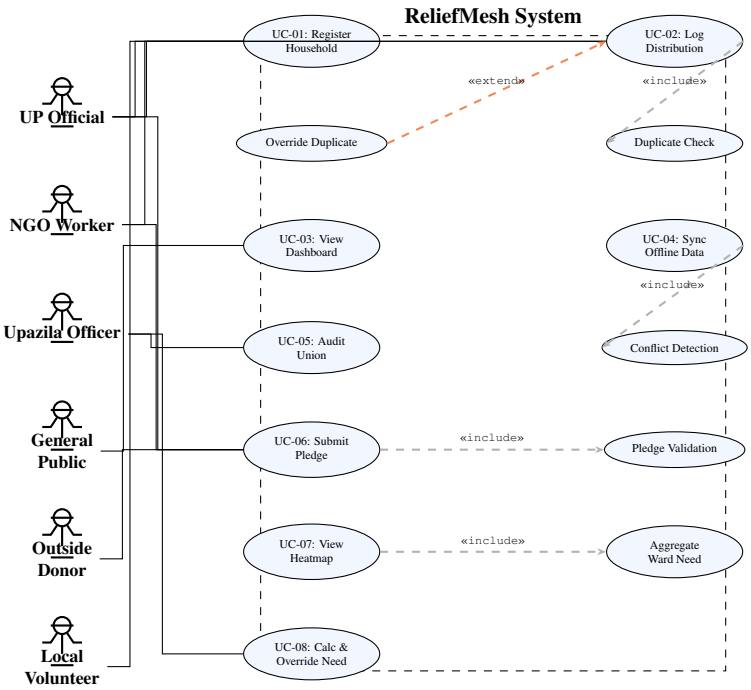

Figure 5: Use Case Diagram

Actors: UP Official, NGO Worker, Upazila Officer, General Public, Outside Individual Donor, Local Volunteer.

Key Use Cases:

- UC-01: Register a Household
- UC-02: Log a Relief Distribution
- UC-03: View Public Dashboard
- UC-04: Sync Offline Data
- UC-05: Audit Union Distributions
- UC-06: Submit Relief Pledge
- UC-07: View Need Heatmap
- UC-08: Calculate & Override Area Need

Relationships: `Log Distribution` **includes** `Duplicate Check`; `Override Duplicate` **extends** `Log Distribution`; `Sync Offline Data` **includes** `Conflict Detection`; `Declare Pledge` **includes** `Pledge Validation`; `View Need Heatmap` **includes** `Aggregate Ward-Level Need`.

4.5 Class Diagram

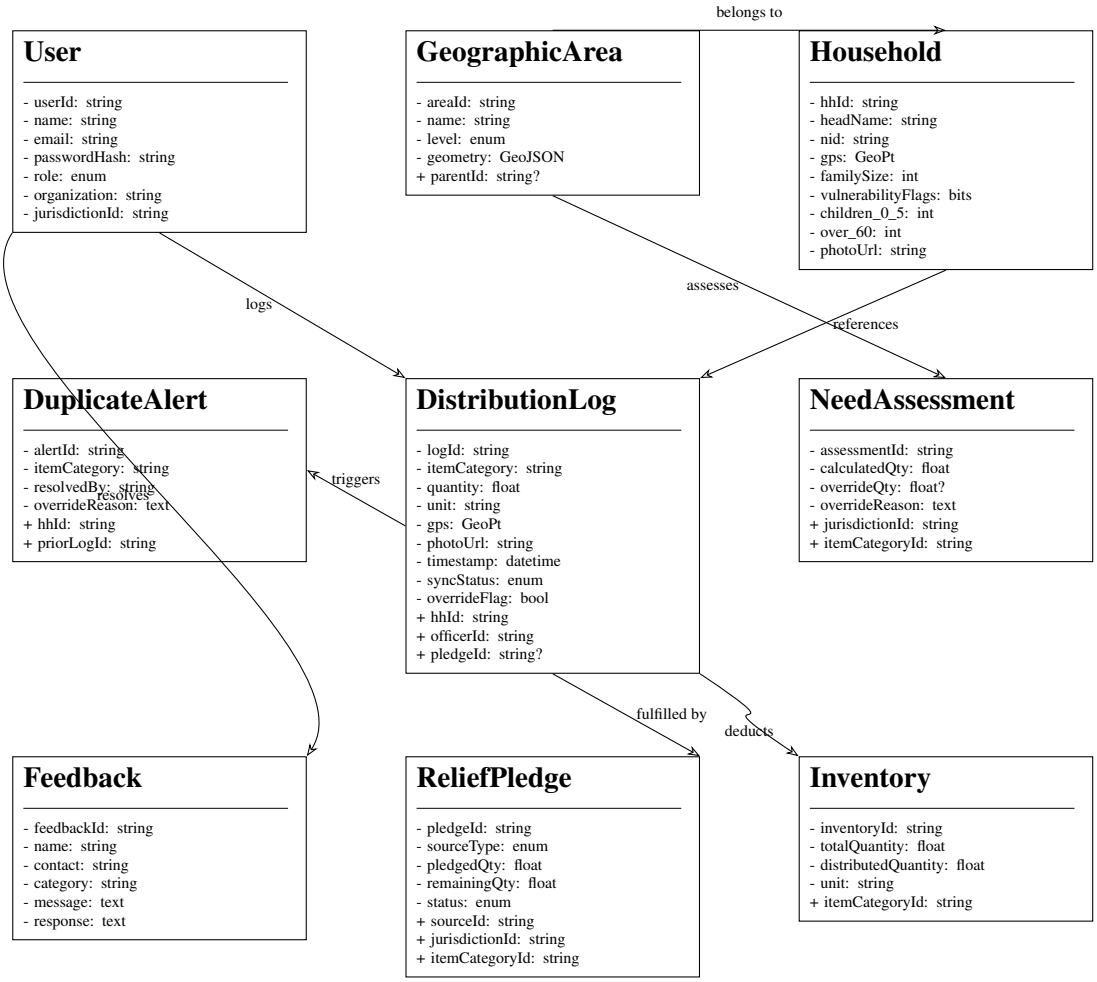

Figure 6: Class Diagram — Core Domain Model

Core Classes:

- **User:** userId, name, email, passwordHash, role, organization, jurisdictionId
- **Household:** hhId, headName, nid, gps, familySize, vulnerabilityFlags, children_0_5, over_60, photoUrl
- **DistributionLog:** logId, hhId, itemCategory, quantity, unit, officerId, gps, photoUrl, timestamp, syncStatus, overrideFlag, pledgeId
- **DuplicateAlert:** alertId, hhId, itemCategory, priorLogId, resolvedBy, overrideReason
- **NeedAssessment:** assessmentId, jurisdictionId, itemCategoryId, calculatedQty, overrideQty, overrideReason
- **ReliefPledge:** pledgeId, sourceId, sourceType, jurisdictionId, itemCategoryId, pledgedQty, remainingQty, status
- **GeographicArea:** areaId, name, level, parentId, geometry
- **Feedback:** feedbackId, name, contact, category, message, response
- **Inventory:** inventoryId, itemCategoryId, totalQuantity, distributedQuantity, unit

Relationships: GeographicArea (self-referencing hierarchy), Household belongs-to GeographicArea, DistributionLog references Household + User + ItemCategory + ReliefPledge (optional), DuplicateAlert references DistributionLog, NeedAssessment references GeographicArea + ItemCategory, ReliefPledge references GeographicArea + ItemCategory.

4.6 Sequence Diagrams

4.6.1 SD-01: Household Registration (Offline)

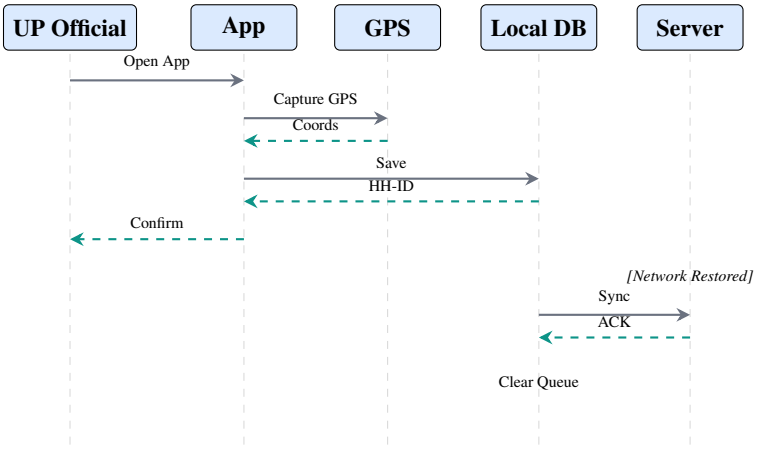

Figure 7: SD-01: Household Registration (Offline)

4.6.2 SD-02: Distribution Log with Duplicate Check

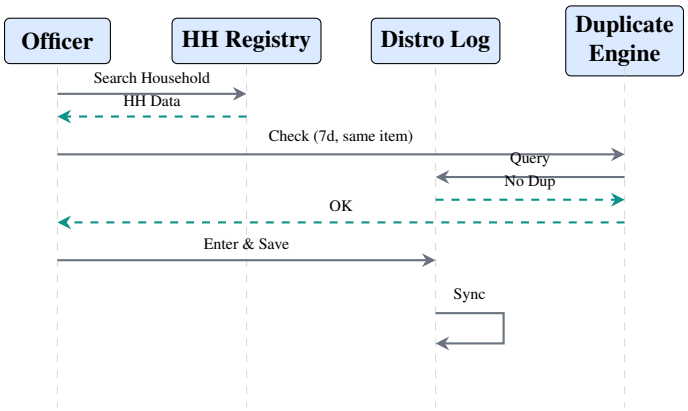

Figure 8: SD-02: Distribution Log with Duplicate Check

4.6.3 SD-03: Sync Conflict Resolution

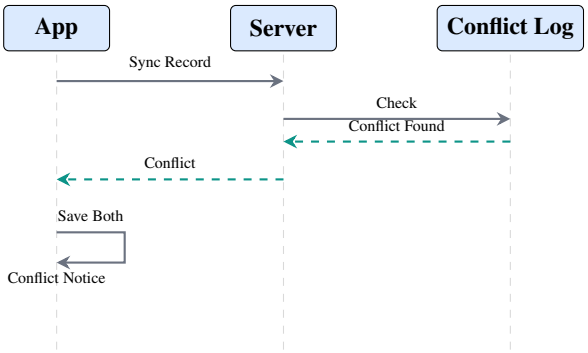

Figure 9: SD-03: Sync Conflict Resolution

4.6.4 SD-04: Need Calculation with Override

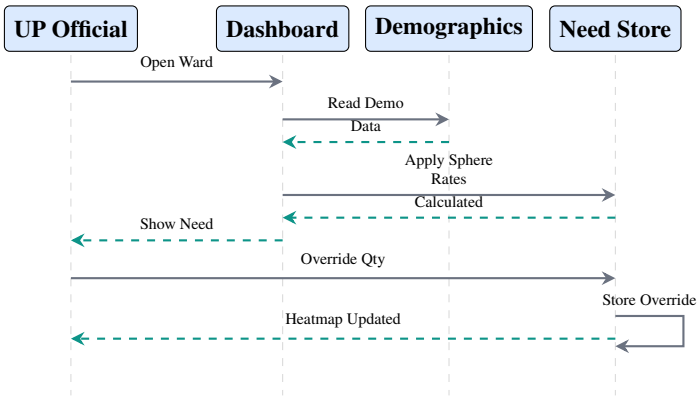

Figure 10: SD-04: Need Calculation with Override

4.6.5 SD-05: Pledge Declaration → Fulfillment

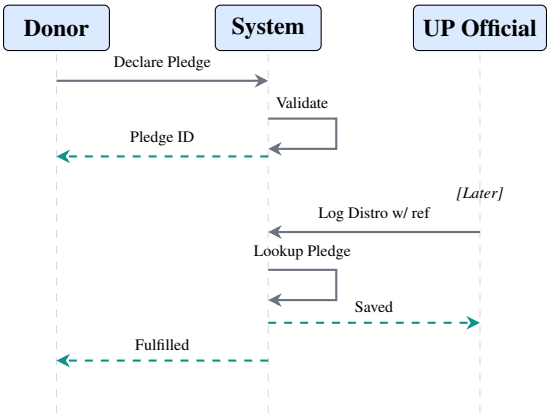

Figure 11: SD-05: Pledge Declaration → Fulfillment

4.7 Activity Diagrams

4.7.1 Relief Distribution Workflow

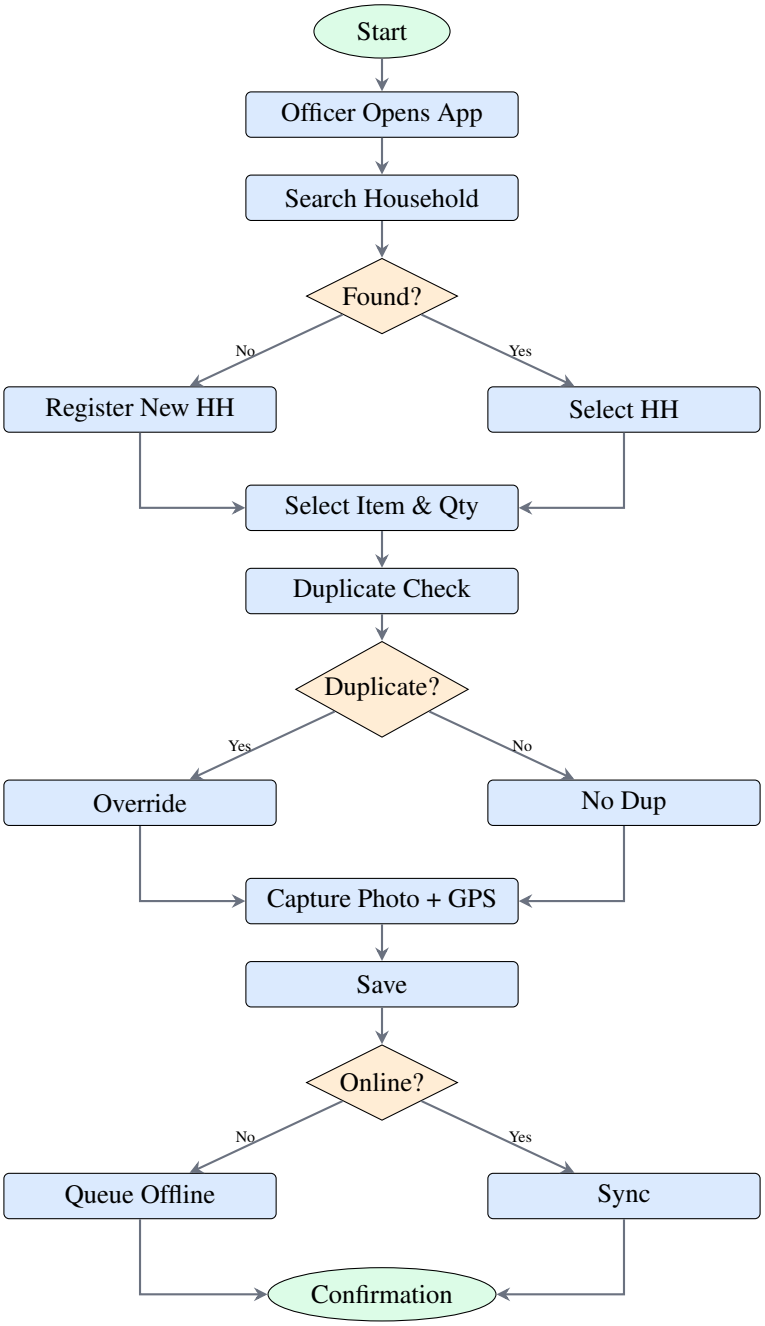

Figure 12: Activity Diagram — Relief Distribution Workflow

4.7.2 Pledge-to-Distribution Workflow

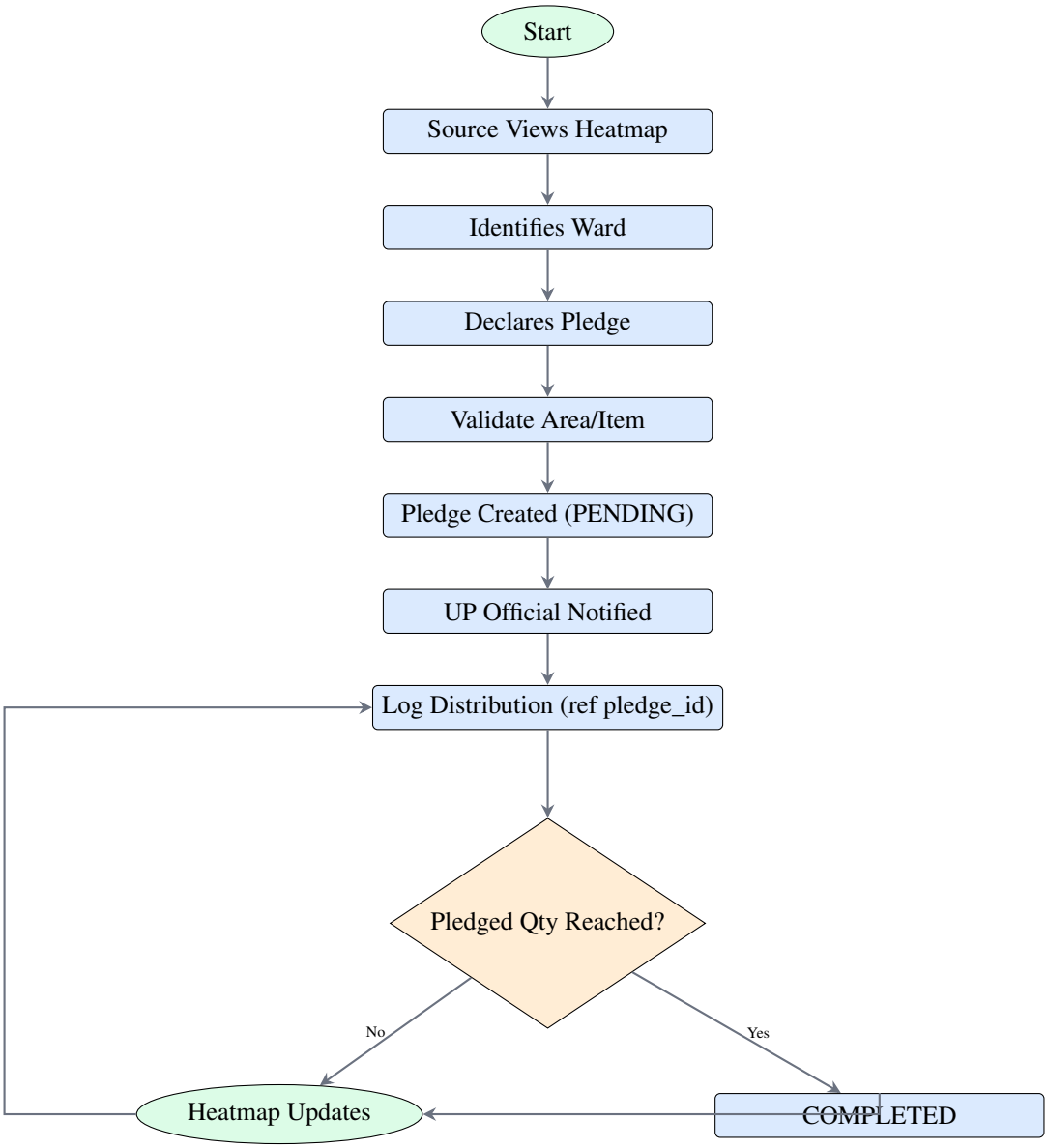

Figure 13: Activity Diagram — Pledge-to-Distribution Workflow

Chapter 5

Database Design (ERD)

5.1 Entity Identification

Table 18: Entity Identification

Entity	Description
User	System accounts — UP officials, Upazila officers, NGO workers
GeographicArea	Full BD administrative hierarchy: Division → District → Upazila → Union → Ward
Household	Registered disaster-affected family unit with age-bracket counts
DistributionLog	Record of one relief distribution (optionally linked to a pledge)
ItemCategory	Lookup table for predefined relief item types with per-person rates
DuplicateAlert	Alert generated when duplicate distribution is detected
SyncConflict	Log of offline sync conflicts pending manual review
Feedback	User-submitted feedback/complaints with admin response
Inventory	Stock tracking per item category
NeedAssessment	Ward-level calculated need per item category, with optional officer override
ReliefPledge	Source declaration of relief supply with status lifecycle

5.2 Entity-Relationship Diagram

The ERD uses Crow's Foot notation with the relationships shown below:

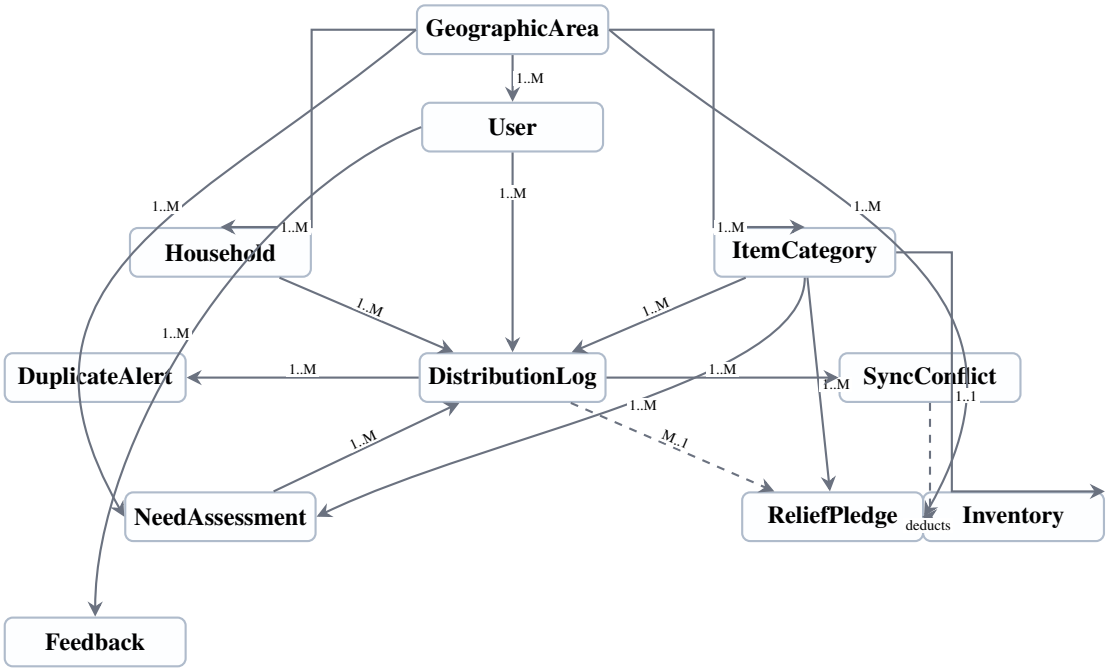

Figure 14: Entity-Relationship Diagram (simplified)

Key Relationships

- **GeographicArea** (1) ⟷ (Many) **User** — one area has many users
- **GeographicArea** (1) ⟷ (Many) **Household** — one union has many households
- **GeographicArea** (self-referencing) — parent_id for hierarchy
- **Household** (1) ⟷ (Many) **DistributionLog** — one household receives many distributions
- **User** (1) ⟷ (Many) **DistributionLog** — one officer logs many distributions
- **DistributionLog** (Many) ⟷ (1) **ReliefPledge** — optional FK for pledge fulfillment
- **ItemCategory** (1) ⟷ (Many) **DistributionLog**, **NeedAssessment**, **ReliefPledge**
- **GeographicArea** (1) ⟷ (Many) **NeedAssessment**, **ReliefPledge**

5.3 Normalization

All tables are in **3NF (Third Normal Form)**:

- **1NF:** All attributes atomic — vulnerability flags are separate boolean columns, age-bracket counts are separate INT columns
- **2NF:** All tables use single-column surrogate PKs (UUID); no composite PKs
- **3NF:** GeographicArea hierarchy is self-referencing; ItemCategory is in its own table; ReliefPledge.remaining_qty is a computed column

5.4 Data Dictionary (Key Tables)

5.4.1 users

- user_id (PK), area_id (FK), name, email (UNIQUE), password_hash, role (ENUM), organization, is_active, created_at

5.4.2 geographic_areas

- area_id (PK), name, level (ENUM: DIVISION/DISTRICT/UPAZILA/UNION/WARD), parent_id (FK, self), geometry (GeoJSON)

5.4.3 households

- hh_id (PK), area_id (FK), head_name, nid (UNIQUE), gps_lat, gps_lng, family_size, is_elderly, is_disabled, is_pregnant, children_0_5, over_60, photo_url, registered_by (FK), created_at

5.4.4 distribution_logs

- log_id (PK), hh_id (FK), officer_id (FK), item_category_id (FK), pledge_id (FK, nullable), quantity, unit, gps_lat, gps_lng, photo_url, timestamp, sync_status (ENUM), is_override, override_reason

5.4.5 need_assessments

- assessment_id (PK), area_id (FK, level=WARD), item_category_id (FK), calculated_qty, override_qty (nullable), override_reason, computed_at, overridden_at, overridden_by (FK)

5.4.6 relief_pledges

- pledge_id (PK), source_id, source_type (ENUM), area_id (FK), item_category_id (FK), pledged_qty, remaining_qty, team_count, volunteer_count, description, status (ENUM: PENDING/IN_FULFILLMENT/CO

Chapter 6

Architecture & Tech Stack

6.1 System Architecture Overview

ReliefMesh uses a **PWA (Progressive Web App) + REST API + MongoDB** three-tier architecture:

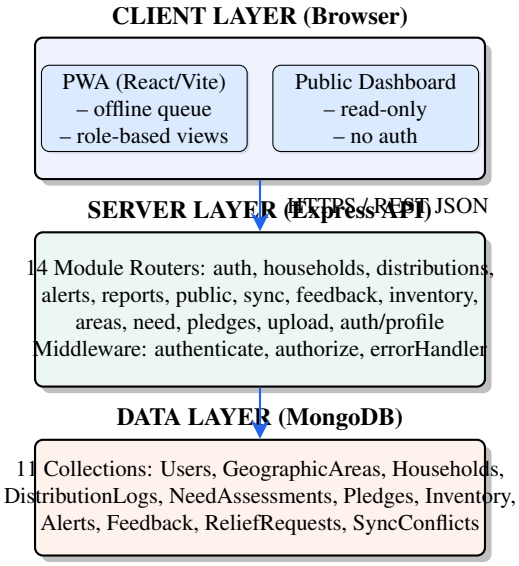

Figure 15: System Architecture Overview (Three-Tier Hybrid Topology)

6.2 Technology Stack

Table 20: Technology Stack

Layer	Technology	Purpose
Frontend	React 18 + Vite 5	SPA with fast HMR
Routing	React Router DOM 6	Client-side routing
Styling	Tailwind CSS 3 + CSS Custom Properties	Utility-first + design tokens
Maps	Leaflet + leaflet.heat + React Leaflet	Interactive heatmap overlays
Offline	localforage (IndexedDB)	Offline queue
PWA	vite-plugin-pwa	Service worker registration
Backend	Express 4 + Node.js 20	REST API server
Database	MongoDB 8 + Mongoose 8	Document DB with schema validation
Auth	JWT (jsonwebtoken) + bcrypt	Token-based authentication
Validation	express-validator	Request validation middleware
Security	helmet + cors + express-rate-limit	HTTP security & rate limiting
Uploads	multer	File upload handling
Reports	PDFKit + json2csv	PDF and CSV generation
Testing	Jest + supertest + mongodb-memory-server	In-memory DB testing

6.3 Offline-First Architecture

Strategy: Single-database offline-first pattern using localforage (IndexedDB) in the browser.

Sync Flow:

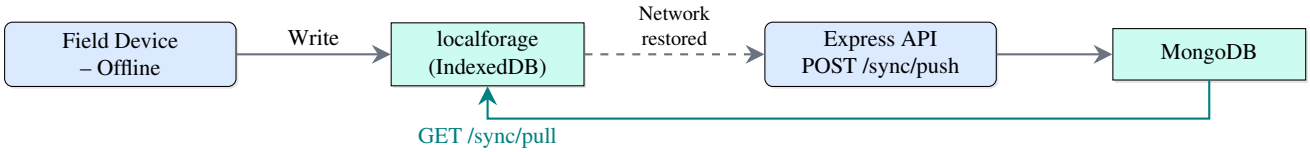

Figure 16: Offline-Sync Data Flow

Conflict Resolution: Last-write-wins by timestamp; conflicting versions saved in ‘sync_conflicts‘ collection for manual review.

6.4 Map & Heatmap Architecture

The map uses a layered rendering stack: Base Tiles (OpenStreetMap) → GeoJSON Layer (boundary polygons) → Heatmap Overlay (leaflet.heat WebGL) → Marker/Info Control (click interaction).

Drill-down flow: District → Upazila → Union → Ward.

Heatmap color scheme: Green (served/need met) → Yellow (partially served) → Red (unmet/critical).

Chapter 7

UI/UX Design

7.1 Information Architecture

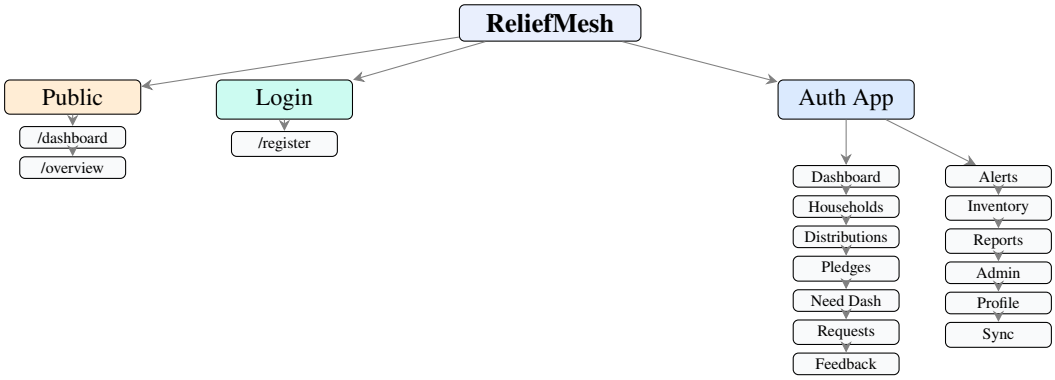

Figure 17: ReliefMesh Information Architecture

7.2 Key User Flows

7.2.1 Flow 1 — Register Household

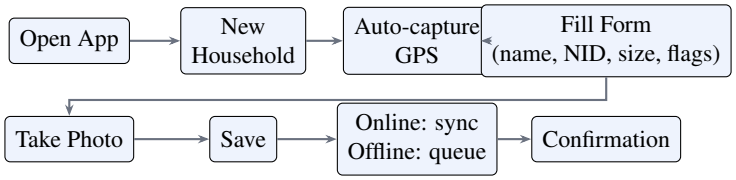

Figure 18: Flow 1 — Register Household

7.2.2 Flow 2 — Log Distribution

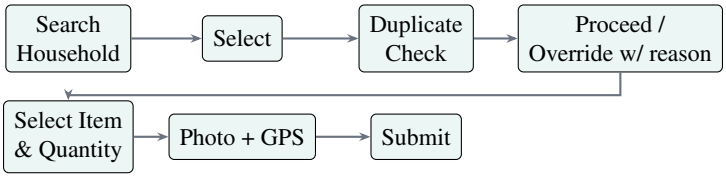

Figure 19: Flow 2 — Log Distribution

7.2.3 Flow 3 — Public Dashboard

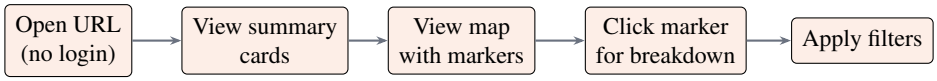

Figure 20: Flow 3 — Public Dashboard

7.2.4 Flow 4 — Calculate Need

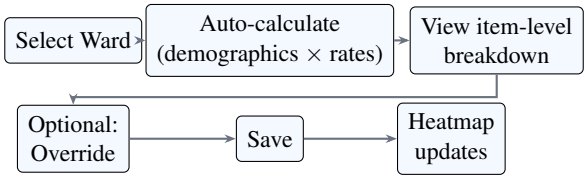

Figure 21: Flow 4 — Calculate Need

7.2.5 Flow 5 — Pledge Source

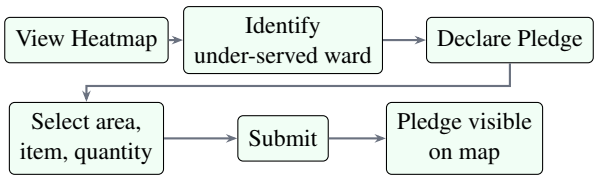

Figure 22: Flow 5 — Pledge Source

7.3 Design System

7.3.1 Color Palette (Result09 Design System)

- **Light Theme:** Primary #2563eb, Surface #ffffff, Text #1e293b, Danger #ef4444, Warning #f59e0b, Success #22c55e
- **Dark Theme:** Surface #0f172a, Card #1e293b, Text #f1f5f9
- **Sidebar:** Dark panel with #0f172a background, #3b82f6 active indicator

7.3.2 Typography

Inter font family (300–900 weights), App Title 22px/700, Section Header 18px/600, Body 16px/400, Caption 13px/400

7.3.3 Component Standards

- Buttons: min height 40px, border-radius 8px, 6 variants
- Input fields: min height 44px, clear label, error/hint/success states
- Cards: 12px border-radius, 1px border, subtle shadow
- Data tables: full-width, striped rows, sticky header
- Sidebar: fixed dark panel, collapsible, mobile overlay
- Topbar: breadcrumbs, theme toggle, sync status indicator

7.4 Accessibility & Low-Literacy Design

Table 22: Accessibility & Low-Literacy Design Decisions

Consideration	Design Decision
Low literacy	Large icons alongside text labels
Bengali support	All field labels in Bengali on field screens
Minimal typing	Dropdowns, GPS auto-capture, photo instead of text
Poor lighting	High contrast UI; 48×48 px minimum tap targets
One-handed use	Primary actions at bottom of screen (thumb zone)
Offline indicator	Persistent status bar showing online/offline/sync

Chapter 8

Implementation Plan

8.1 Phase 1: Core Modules (Weeks 1–16)

Table 24: Phase 1: Core Modules

#	Module	Owner	Status
M1	Project setup (repo, tooling)	Kamrul	✓ Complete
M2	MongoDB schemas + Mongoose models	Kamrul	✓ Complete
M3	Authentication API (register, login, JWT)	Kamrul	✓ Complete
M4	Household registration (API + frontend)	Kamrul, Sayeda	✓ Complete
M5	Offline support — IndexedDB (localforage)	Kamrul	✓ Complete
M6	Distribution log (API + frontend)	Kamrul, Sayeda	✓ Complete
M7	Duplicate detection engine	Kamrul	✓ Complete
M8	Duplicate alert UI + override	Sayeda, Nahid	✓ Complete
M9	Sync engine (push/pull) + conflict log	Kamrul	✓ Complete
M10	Upazila Officer dashboard	Sayeda	✓ Complete
M11	Public dashboard + map view	Nahid	✓ Complete
M12	Report export (PDF/CSV)	Kamrul	✓ Complete
M13	Testing (unit + integration + UAT)	Abidul	✓ Complete
M14	Bug fixes + polish	All	✓ Complete
M15	Demo video + presentation prep	Nahid, Abid	✓ Complete
M16	Feedback module	Kamrul, Sayeda	✓ Complete
M17	Inventory/stock tracking	Kamrul	✓ Complete
M18	User profile management	Kamrul, Nahid	✓ Complete
M19	Pagination + search	Kamrul	✓ Complete
M20	Enhanced dashboard	Kamrul	✓ Complete

8.2 Phase 2: v2 Features (Post-Semester)

Table 26: Phase 2: v2 Features

#	Module	Owner	Priority
M21	GeographicArea model + boundary API	Kamrul	High
M22	ItemCategory: per_person_per_day_qty	Kamrul	High
M23	Household: age-bracket fields	Kamrul	High
M24	Need calculation engine + API	Kamrul	High
M25	Need dashboard UI	Sayeda, Nahid	High
M26	Pledge model + CRUD API	Kamrul	High
M27	Pledge UI	Sayeda	High
M28	Heatmap rendering	Nahid	High
M29	Pledge-Distribution linking	Kamrul	Medium
M30	Auto-update pledge remaining_qty	Kamrul	Medium
M31	E2E testing for need + pledge	Abidul	High
M32	Volunteer registration + matching	All	Medium

8.3 Development Environment

Prerequisites: Node.js $\geq$ 20.x LTS, npm $\geq$ 10.x, MongoDB $\geq$ 8.x

Setup:

```
git clone https://github.com/Team-Skipper/reliefmesh.git
cd backend && cp ../.env.example .env && npm install && npm run seed && npm run
    dev
cd ../frontend && npm install && npm run dev
```

8.4 Coding Standards

- Language: JavaScript (ES2022+), no TypeScript (team familiarity)
- Linter: ESLint with Airbnb config
- Formatter: Prettier (2-space indent, single quotes)
- Files: kebab-case, Functions: camelCase, Components: PascalCase, DB: snake_case
- All async operations use `async/await`

8.5 Git Branching Strategy

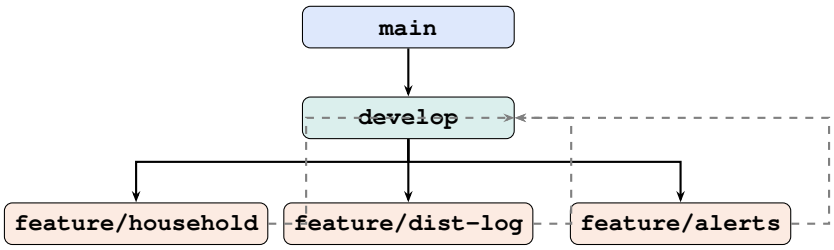

Figure 23: Git Branching Strategy — Git Flow Adapted

Rules: No direct commits to `main`, PRs require 1 reviewer, daily commits, commit types: `feat:`, `fix:`, `docs:`, `style:`, `test:`, `chore:`

Chapter 9

Testing & QA

9.1 Test Plan

Objective: Verify all functional requirements (FR-01 to FR-44), validate NFRs, ensure RBAC enforcement, confirm offline sync and duplicate detection.

Scope: Unit testing, Integration testing, System testing, User Acceptance Testing (UAT).

Tools: Jest, Supertest, React Testing Library, Postman, Chrome DevTools, Lighthouse.

9.2 Unit Test Cases (29 Passing)

- **Authentication (TC-01 to TC-04):** Valid login returns JWT ✓, Invalid password rejected ✓, Non-existent user rejected ✓, Role embedded in JWT ✓
- **Household (TC-05 to TC-08):** Valid registration saves HH-ID ✓, Duplicate NID rejected ✓, Missing field fails validation ✓, GPS stored correctly ✓
- **Distribution (TC-09 to TC-11):** Valid log saved ✓, Unknown HH-ID rejected ✓, Quantity must be positive ✓
- **Duplicate Detection (TC-12 to TC-16):** Same item within 7 days → duplicate ✓, Same item after 8 days → no duplicate □, Different item → no duplicate ✓, Override accepted with reason ✓, Override rejected without reason ✓

9.3 Integration Test Cases (7 Passing)

Table 28: Integration Test Cases

TC	Flow	Status
TC-17	Register → Log → Duplicate	✓
TC-18	Offline queue → sync	□ (manual)
TC-19	Role access control	✓
TC-20	Public dashboard (no auth)	✓
TC-21	Upazila jurisdiction filter	□
TC-22	Conflict detection on sync	✓

9.4 Module-Specific Tests (Additional)

Feedback (TC-FB01 to FB04): Submit without auth ✓, List with auth ✓, Respond ✓, Submit without name rejected ✓

Inventory (TC-INV01 to INV03): Create ✓, List ✓, UP Official cannot create ✓

Need Calculation & Pledge (v2): 10 test cases planned (TC-NEED01 to NEED05, TC-PLEDGE01 to PLEDGE05)

9.5 UAT Scenarios

Table 30: UAT Scenarios

Scenario	Description
UAT-01	Register 5 households offline → sync → verify in MongoDB
UAT-02	Attempt duplicate distribution → warning shown → override
UAT-03	Upazila audit: view all distributions → export PDF
UAT-04	Public transparency: dashboard loads without login, no PII
UAT-05	Need assessment: run calculation → verify heatmap → override
UAT-06	Pledge-to-fulfillment: donor pledges → official distributes → status updates

9.6 Test Results Summary

Table 32: Test Results Summary

Category	Total	Passed	Failed
Unit Tests (Phase 1)	29	29	0
Integration Tests	7	7	0
Module Tests (Feedback, Inventory)	7	7	0
v2 Tests (Planned)	10	—	—
Total	**53**	**43**	**0**

Chapter 10

Security & Access Control

10.1 Threat Model (STRIDE)

Table 34: STRIDE Threat Model

Threat	Category	Mitigation
Fake officer identity	Spoofing	JWT + bcrypt password hashing
Modify distribution log	Tampering	Logs immutable once synced
Deny making a log entry	Repudiation	All logs include officer_id + timestamp + GPS
Expose household PII	Information Disclosure	PII only in private endpoints; public data aggregated
Flood API with requests	DoS	Rate limiting (express-rate-limit)
Elevate privilege	Elevation of Privilege	RBAC on every protected route
Falsify pledge data	Spoofing	Public pledges require email; admin requires auth
Tamper need calculation	Tampering	Override logged with officer_id + timestamp + reason

10.2 Authentication Mechanism

Method: JWT (HS256, 7-day expiry, secret in `.env`)

Login Flow:

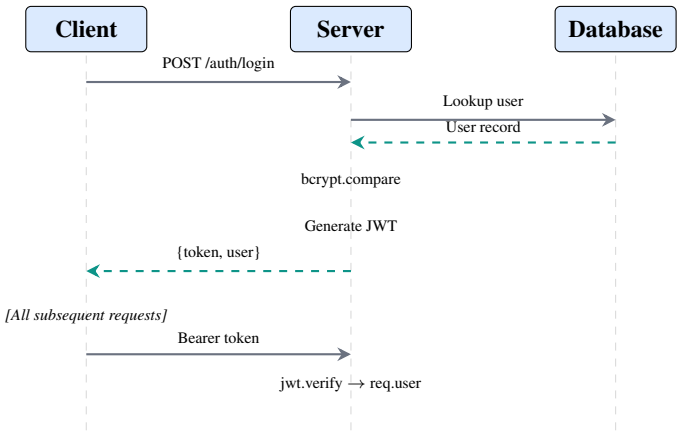

Figure 24: Login Flow — JWT Authentication Sequence

JWT Payload: `{ sub: "user-uuid", role: "UP_OFFICIAL", jurisdiction_id: "union-uuid", iat, exp }`

10.3 Role-Based Access Control

Roles & Core Permissions:

Table 36: Roles & Permissions Matrix

Permission	Public	UP Official	NGO	Upazila Officer
View public dashboard	✓	✓	✓	✓
Register household	×	✓	×	×
Log distribution	×	✓	✓	×
Override duplicate	×	✓	✓	×
View all unions	×	×	×	✓
Export reports	×	×	×	✓
Manage users	×	×	×	✓
Override need assessment	×	✓	×	✓
Declare pledge	×	✓	✓	✓

Jurisdiction Enforcement: Every data query is filtered by `jurisdiction_id`. A UP Official can only access their union's data; an Upazila Officer can access all unions under their Upazila.

10.4 Data Privacy Design

PII Handling: NID numbers are Sensitive PII — visible only to authenticated users in the same jurisdiction, never in public responses. GPS coordinates are approximate in reports.

Public Dashboard Rules: Only aggregated counts per union (no per-household breakdown), no NID/name/GPS in public responses, pre-computed and cached endpoint.

10.5 Input Validation & Injection Prevention

All inputs validated server-side with `express-validator` (trim, length checks, regex patterns). No raw SQL string concatenation — parameterized queries throughout. React escapes output by default. Security headers via `helmet` middleware. Rate limiting: 200 requests per 15 minutes per IP.

Chapter 11

Deployment & Maintenance

11.1 Deployment Overview

Table 38: Deployment Overview

Component	Platform	Tier
Frontend (React PWA)	Netlify	Free
Backend (Node.js API)	Railway	Free / Hobby
MongoDB	MongoDB Atlas	Free (512MB)
Photo storage	Local FS / Cloudinary	Local for prototype
Domain	Netlify default subdomain	Free

11.2 Deployment Steps

11.2.1 Prepare Repository

`.env` in `.gitignore`, verify build works locally.

11.2.2 Set Up MongoDB Atlas

Create free M0 cluster, add database user, whitelist IP `0.0.0.0/0`, copy connection string.

11.2.3 Deploy Backend to Railway

New Project → Deploy from GitHub → Set environment variables (`MONGODB_URI`, `JWT_SECRET`, `NODE_ENV=production`).

11.2.4 Deploy Frontend to Netlify

New site → Import from GitHub → Build command `npm run build`, Publish directory `dist`, Set `VITE_API_BASE_URL`.

11.2.5 Verify Deployment

- x API health check: `GET /v1/health` → `{ status: "ok" }`
- x Frontend loads at Netlify URL
- x Login works with seeded test accounts
- x Household registration → MongoDB → dashboard
- x Public dashboard loads without login
- x Feedback submission, inventory, profile, pagination all working

11.3 Environment Variables

Table 40: Environment Variables

Variable	Description
MONGODB_URI	MongoDB connection string
JWT_SECRET	Signing key for JWTs (min 32 chars)
JWT_EXPIRES_IN	Token lifetime (e.g., 7d)
CLOUDINARY_*	Cloudinary credentials (optional)
NODE_ENV	development or production
PORT	Server port
VITE_API_BASE_URL	Frontend API base URL

11.4 Backup & Recovery

Database: MongoDB Atlas automated backups (paid tier) or weekly manual mongodump.

Recovery: mongorestore then npm run db:status to verify.

11.5 Future Roadmap

Table 42: Future Roadmap

Feature	Priority
NID API integration (national DB)	High
Real-time WebSocket sync	Medium
Native Android app (React Native)	Medium
Multi-district multi-disaster support	High
End-to-end encryption of PII	High
SMS notification for households	Medium
TopoJSON boundary compression	Medium
Automated pledge-donor matching	Medium

Chapter 12

Project Management

12.1 Meeting Minutes Log

Table 44: Meeting Minutes Log

#	Date	Type	Key Decisions
1	2026-05-27	Team kickoff	Project topic confirmed — ReliefMesh
2	2026-05-28	Supervisor	Approved team formation; feedback on scope
3	2026-06-01	Team	Architecture finalized (Express + Mongoose + React/Vite)
4	2026-06-05	Team	Core features built; UI restyled with Result09 design system
5	2026-06-09	Team	Offline queue wired; sync completed; all 28 tests passing
6	2026-06-10	Team	Feedback, inventory, profile, pagination built; 36 tests passing
7	2026-06-20	Team	v2 features approved: need calc, pledge, heatmap

12.2 Weekly Progress Summary

Table 46: Weekly Progress Summary

Week	Dates	Key Achievements
1	May 25–31	Project setup, repo structure, documentation drafts (3.1–3.7)
2	Jun 1–7	All backend modules implemented; all frontend pages built
3	Jun 8–14	Offline queue, sync service, photo upload; 28 tests passing
4	Jun 15–21	Feedback, inventory, profile, pagination; 36 tests passing; all 14 docs updated

12.3 Git Discipline

- No direct commits to `main`
- All features via pull requests with ≥ 1 reviewer
- Meaningful commit messages (`feat:`, `fix:`, `docs:`, etc.)
- Branches: `main` → `develop` → `feature/xxx`

12.4 Change Request Log

Table 48: Change Request Log

CR ID	Date	Description	Status
CR-001	—	Add SMS notification feature	Rejected (out of scope)
CR-002	2026-06-20	v2 features: need calc, pledge, heatmap, geographic hierarchy	Approved

12.5 RACI Chart

Table 50: RACI Chart

Deliverable	Kamrul	Abidul	Sayeda	Nahid
SRS Document	C	**R**	C	I
DFD / UML / ERD	C	**R**	I	I
Wireframes & Mockups	I	C	**R**	C
Frontend Implementation	C	I	**R**	C
Backend / Database	**R**	C	I	I
Test Plan & Test Cases	C	**R**	I	I
Demo Video	I	I	C	**R**
Presentation Slides	C	**R**	C	I

Key: R = Responsible, A = Accountable, C = Consulted, I = Informed

Chapter 13

References & Bibliography

All references follow **IEEE citation style**.

13.1 Academic & Research References

[1] M. M. Uddin, M. S. Islam, and M. A. Rahman, "Factors affecting disaster coordination between government and NGOs for relief and rehabilitation activities in Bangladesh," *Indian J. Humanities Soc. Sci.*, vol. 2, no. 10, pp. 8–18, 2021.

[2] M. Islam *et al.*, "Disaster relief and rehabilitation at the local level in Bangladesh: assessing the practices of Union Parishads," *J. Disaster Sci. Manage.*, Springer Nature, Jan. 2026.

[3] Harvard Humanitarian Initiative, "Development, Governance, and Disaster Nexus: Bangladesh Viewpoint," 2025. [Online]. Available: `https://hhi.harvard.edu`

[4] S. Hossain and M. N. Islam, "Corruption in cyclone relief and reconstruction: Evidence from a public fund distribution in Bangladesh," *J. Development Studies*, Taylor & Francis, 2025.

[5] M. A. Hossain, "Disaster management discourse in Bangladesh," in *Disaster Management*, InTechOpen, 2013, ch. 4.

[6] UNDRR, "Bridging national strategy and local action: Bangladesh's success in vertical DRR integration," 2025.

[7] M. A. Rahman, M. Hossain, and S. Sultana, "Disaster management in Bangladesh: Developing an effective emergency supply chain network," *Annals of Operations Research*, vol. 283, pp. 1463–1487, 2018.

[8] A. Islam and M. R. Islam, "Flood hazard assessment and monitoring in Bangladesh," *Earth*, vol. 6, no. 3, Aug. 2025.

[9] M. K. Ahmed, "Towards cyclone resilience: Examining local challenges in shelter management in Southwest Bangladesh," *Int. J. Disaster Risk Reduction*, Dec. 2025.

[10] M. A. Uddin, "Disaster risk reduction in Bangladesh: A comparison of three major floods," *Int. J. Disaster Risk Reduction*, vol. 96, Oct. 2023.

13.2 Technical References

[11] MongoDB Inc., "MongoDB Documentation," v8.0. [Online]. Available: `https://www.mongodb.com/docs/`

[12] localForage Contributors, "localForage: Offline Storage Library," Mozilla. [Online]. Available: `https://localforage.github.io/localForage/`

[13] OpenJS Foundation, "Node.js Documentation," v20 LTS. [Online]. Available: `https://nodejs.org/en/docs`

[14] Facebook Inc., "React Documentation," v18. [Online]. Available: `https://react.dev`

[15] Vite Contributors, "Vite Documentation," v5.x. [Online]. Available: `https://vitejs.dev/guide/`

[16] MongoDB Inc., "Mongoose ODM Documentation," v8.x. [Online]. Available: `https://mongoosejs.com/docs/`

[17] M. D. Srinath, "STRIDE Threat Modeling," Microsoft SDL, 2007.

[18] OWASP Foundation, “OWASP Top Ten 2021.” [Online]. Available: https://owasp.org/Top10/

13.3 Standards & Frameworks

[19] IEEE Std 830-1998, “IEEE Recommended Practice for Software Requirements Specifications.”

[20] Sendai Framework for Disaster Risk Reduction 2015–2030, UNDRR, 2015.

[21] Government of Bangladesh, “Disaster Management Act 2012,” Ministry of Disaster Management and Relief.

[22] Sphere Association, *The Sphere Handbook: Humanitarian Charter and Minimum Standards in Humanitarian Response*, 4th ed., 2018.

[23] V. Agafonkin, “Leaflet.heat — Heatmap plugin for Leaflet,” GitHub.

[24] HeiGIT gGmbH, “openrouteservice: OpenStreetMap-based routing API.”

[25] Bangladesh Bureau of Statistics, “Population and Housing Census 2022.”

[26] ADPC and UNDP, “Comprehensive Disaster Management Programme (CDMP) Bangladesh,” 2019.

[27] M. B. Hossain and S. Rahman, “Community-based flood vulnerability assessment in Feni District, Bangladesh,” *J. Bangladesh Studies*, vol. 24, no. 1, pp. 45–62, 2024.

[28] GeoBoundaries, “Bangladesh Administrative Boundaries (ADM0–ADM4),” 2025.

Chapter 14

Presentation & Defense

14.1 Presentation Structure

Table 52: Presentation Structure

Slide	Time (min)	Content
1	0:30	Title + Team introduction
2	1:00	Problem statement (current system pains)
3	1:30	Objective + Scope
4	1:00	System architecture (Context + Tech stack)
5	2:00	Live demo walkthrough
6	1:00	Key features: Offline, Duplicate detection, Sync
7	1:00	Key features: Need assessment, Pledge, Heatmap
8	1:00	Security & Access Control
9	1:00	Testing summary
10	0:30	Future roadmap
11	1:00	Q&A

14.2 Live Demo Walkthrough

Total duration: 7–10 minutes.

1. **Public Dashboard** (0:30) — Open localhost URL, no login required
2. **Login & Role** (0:30) — Log in as UP Official, observe dashboard
3. **Register Household** (1:00) — Create a new household, show GPS + photo capability
4. **Log Distribution** (1:00) — Search household, distribute item, capture photo
5. **Duplicate Alert** (1:00) — Attempt same-day duplicate → warning shown → demonstrate override with reason
6. **Offline & Sync** (1:00) — Disconnect WiFi → operate offline → reconnect → sync and verify
7. **Upazila Dashboard** (0:30) — Login as Upazila Officer, show cross-union analytics
8. **Reports** (0:30) — Export report (PDF/CSV)
9. **Feedback** (0:30) — Submit feedback, show admin response
10. **Inventory** (0:30) — View, create, permission enforcement
11. **Need Dashboard & Pledge (v2)** (1:00) — Run calculation, show heatmap, declare pledge

14.3 Key Talking Points

- **Offline-first:** Real-world relief zones have no internet. ReliefMesh works fully offline and syncs when connectivity returns.
- **Duplicate prevention:** 7-day lookback prevents double-counting. Override requires a reason (audit trail).
- **Transparency:** Public dashboard shows real-time allocation data. No login needed. Aggregated, no PII.
- **Need-based:** v2 shifts from "who asked?" to "who needs most?" using Sphere standards + local demographics.

- **Accountability:** Every action is logged — who, what, when, where (GPS), why (reason).

14.4 Common Q&A Preparation

- **Q:** How does offline work? — **A:** localforage stores data in IndexedDB. Sync engine pushes/pulls on reconnect with conflict resolution.
- **Q:** What about data security? — **A:** JWT with bcrypt, RBAC, rate limiting, helmet headers, PII never exposed in public endpoints.
- **Q:** Can this scale? — **A:** MongoDB sharding + horizontal API scaling. Designed for Union $\rightarrow$ Upazila $\rightarrow$ District hierarchy.
- **Q:** How is this different from paper-based systems? — **A:** Real-time visibility, duplicate prevention, GPS audit trail, automated reports.
- **Q:** What is the tech stack? — **A:** MERN stack (MongoDB, Express, React, Node.js) with localforage for offline, Leaflet for maps, Vite for build.

Appendix A

Project Repository Structure

A.1 Repository Layout

```
ReliefMesh/
|-- backend/
|   |-- modules/                # 14 feature modules
|   |   |-- auth/               # Authentication & authorization
|   |   |-- alerts/             # Alert management
|   |   |-- areas/              # Geographic area hierarchy
|   |   |-- distributions/      # Relief distribution logging
|   |   |-- feedback/           # Feedback & response
|   |   |-- households/         # Household registration
|   |   |-- inventory/          # Stock/inventory tracking
|   |   |-- needCalc/           # Need assessment calculation
|   |   |-- pledges/            # Pledge/commitment tracking
|   |   |-- public/             # Public-facing endpoints
|   |   |-- reliefRequests/     # Citizen relief requests
|   |   |-- reports/            # Report generation (PDF/CSV)
|   |   |-- sync/               # Offline sync engine
|   |   `-- uploads/            # Photo upload handling
|   |-- middleware/              # authenticate, authorize, errorHandler, upl
|   |-- config/                 # Environment configuration
|   |-- db/                     # Seeds & database setup
|   |-- tests/                  # Test helpers (setup, teardown, helpers)
|   `-- server.js               # App entry point
|-- frontend/
|   |-- src/
|   |   |-- modules/            # 12 feature modules (parallel to backend)
|   |   |-- components/         # Reusable UI
|   |   |   |-- common/         # Shared UI elements
|   |   |   |-- layout/         # Sidebar, Topbar, PageShell
|   |   |   |-- maps/           # Leaflet map components
|   |   |   |-- tables/         # Data tables
|   |   |   `-- ui/             # Buttons, inputs, cards, etc.
|   |   |-- services/           # API client, offline sync
|   |   |-- hooks/              # Custom React hooks
|   |   |-- styles/             # Design tokens (tokens.css, base.css,...)
|   |   `-- constants/          # Navigation, enums
|   |-- App.jsx                 # Root component
|   `-- main.jsx                # Entry point
|-- documentation/              # 14 section markdown files (3.1--3.14)
|-- diagrams/                   # draw.io files (DFD, UML, ERD, architecture)
|-- designs/                    # Figma exports (wireframes, mockups)
|-- reports/                    # Meeting minutes, weekly progress
|-- submission/                 # Demo video, slides, consolidated report
|-- assets/                     # Shared media
|-- tex/                        # LaTeX source for this document
```

```
|   |-- sections/               # Per-section .tex files
|   `-- appendices/             # Appendix .tex files
`-- README.md                   # Project overview
```

Appendix B

API Endpoint Summary

B.1 Complete API Endpoint Inventory

Total: 50+ endpoints across 14 modules.

Table 54: API Endpoint Summary

Module	Key Endpoints
Auth	`POST /login`, `POST /register`, `GET /me`, `PUT /me`, `POST /change-password`
Households	CRUD + `GET /stats/overview`
Areas	`GET /`, `GET /hierarchy`, `GET /:id`, `GET /:id/children`
Distributions	Full CRUD
Alerts	`GET /`, `PUT /:id/resolve`
Need	`GET /heatmap` (public), CRUD + `POST /calculate`, `PUT /:id/override`
Pledges	`GET /my`, CRUD + `PUT /:id/status`
Inventory	CRUD
Feedback	`POST /` (public), `GET /`, `PUT /:id/respond`
ReliefRequests	Citizen: `create/mine/cancel`; Admin: `list/review`
Reports	`GET /households`, `/distributions`, `/needs` (PDF/CSV)
Public	`GET /dashboard`, `/map`, `/item-categories`, `/activities`, `/heatmap`
Sync	`POST /push`, `GET /pull?since=`
Upload	`POST /image`

B.2 Sample API Responses

Public Dashboard (GET /api/v1/dashboard):

```
{
  "totalHouseholds": 42,
  "totalDistributed": 256,
  "lastSync": "2026-06-09T18:30:00Z",
  "distributionByUnion": [
    { "union": "South Kattali", "count": 120 },
    { "union": "North Kattali", "count": 136 }
  ]
}
```

Login (POST /api/v1/auth/login):

```
{
  "token": "eyJhbGciOiJIUzI1NiIs...",
  "user": {
    "_id": "uuid",
    "name": "UP Official",
    "role": "UP_OFFICIAL",
    "jurisdiction_id": "union-uuid",
    "jurisdiction_name": "South Kattali"
```

```
    }
}
```

Appendix C

Key Diagram References

C.1 Diagram Inventory

Table 55: Key Diagram References

Diagram	File Location
Context Diagram (Level 0 DFD)	`diagrams/context-diagram.drawio`
Level 1 DFD	`diagrams/level-1-dfd.drawio`
Use Case Diagram	`diagrams/usecase-diagram.drawio`
Class Diagram	`diagrams/class-diagram.drawio`
ER Diagram	`diagrams/erd.drawio`
Architecture Diagram	`diagrams/architecture-diagram.drawio`
Activity Diagrams	`diagrams/activity-diagrams/`
Sequence Diagrams	`diagrams/sequence-diagrams/`

C.2 Tool Chain

Table 57: Tool Chain

Tool	Purpose
draw.io (diagrams.net)	DFD, UML, ERD, architecture
Figma	UI/UX wireframes and mockups
Canva	Presentation slides
Leaflet + OpenStreetMap	Interactive maps
Postman	API testing
Jest + Supertest	Unit and integration testing
Git + GitHub	Version control
Netlify	Frontend hosting
Railway	Backend hosting
MongoDB Atlas	Database hosting

End of ReliefMesh 14-Module Consolidated Report

Team_Skipper | CSE-3208 | Rangamati Science and Technology University (RMSTU)